Operating Systems Laboratory

CPU Scheduling Algorithms — C Programs

3rd Semester CSE (AI & ML) • Student-friendly programs with sample outputs

Learning objective: Understand how the CPU selects processes and calculate CT, TAT and WT. The programs below are intentionally simple for classroom demonstration.

Core formulas

CT = Completion Time TAT = CT − AT WT = TAT − BT

Algorithm	Type	Basic rule
FCFS	Non-preemptive	First process in arrival order runs first
SJF	Non-preemptive	Shortest available burst runs first
Priority	Non-preemptive	Highest-priority process runs first
SRTF	Preemptive	Shortest remaining CPU time runs first
Round Robin	Preemptive	Each process gets a fixed time quantum in turn

Priority convention in this guide: smaller number = higher priority. The SJF program assumes all processes arrive at time 0. SRTF and Round Robin include Arrival Time to show the scheduling process more clearly.

Note on CFS

CFS (Completely Fair Scheduler) is a Linux kernel scheduler concept. It is not appropriate to reproduce the real kernel CFS as a simple undergraduate C program. Students should first understand the standard scheduling algorithms below, then study CFS conceptually.

1. FCFS — First Come, First Served

Concept: FCFS executes in arrival order.

Sample input: Input: n=3, BT: 5 3 2

C Program

```
#include <stdio.h>

int main()
{
    int n, i;
    int bt[10], ct[10], tat[10], wt[10];
    float awt = 0, atat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("Enter Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    ct[0] = bt[0];

    for (i = 1; i < n; i++)
        ct[i] = ct[i - 1] + bt[i];

    for (i = 0; i < n; i++)
    {
        tat[i] = ct[i];       // AT = 0
        wt[i] = tat[i] - bt[i];
        awt += wt[i];
        atat += tat[i];
    }

    printf("\nP\tBT\tCT\tTAT\tWT\n");
    for (i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\t%d\n",
               i + 1, bt[i], ct[i], tat[i], wt[i]);

    printf("\nAverage WT = %.2f\n", awt / n);
    printf("Average TAT = %.2f\n", atat / n);

    return 0;
}
```

Compile and execute

```
gcc fcfs.c -o fcfs
./fcfs
```

Correct sample output:

```
Output:
P■BT■CT■TAT■WT
P1■5■5■5■0
P2■3■8■8■5
P3■2■10■10■8

Average WT = 4.33
Average TAT = 7.67
```

How to explain the result: Order: P1 → P2 → P3. Gantt: | P1 | P2 | P3 |

2. SJF — Shortest Job First (Non-preemptive)

Concept: Select the shortest available burst.

Sample input: Input: n=3, BT: P1=6, P2=2, P3=4

C Program

```
#include <stdio.h>

int main()
{
    int n, i, j, temp;
    int bt[10], pid[10], ct[10], tat[10], wt[10];
    float awt = 0, atat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        pid[i] = i + 1;
        printf("Enter Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    /* Sort by shortest Burst Time */
    for (i = 0; i < n - 1; i++)
        for (j = i + 1; j < n; j++)
            if (bt[i] > bt[j])
            {
                temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
                temp = pid[i]; pid[i] = pid[j]; pid[j] = temp;
            }

    ct[0] = bt[0];
    for (i = 1; i < n; i++)
        ct[i] = ct[i - 1] + bt[i];

    for (i = 0; i < n; i++)
    {
        tat[i] = ct[i];       // AT = 0
        wt[i] = tat[i] - bt[i];
        awt += wt[i];
        atat += tat[i];
    }

    printf("\nP\tBT\tCT\tTAT\tWT\n");
    for (i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\t%d\n",
               pid[i], bt[i], ct[i], tat[i], wt[i]);

    printf("\nAverage WT = %.2f\n", awt / n);
    printf("Average TAT = %.2f\n", atat / n);

    return 0;
}
```

Compile and execute

```
gcc sjf.c -o sjf
./sjf
```

Correct sample output:

```
Output:
P■BT■CT■TAT■WT
P2■2■2■2■0
P3■4■6■6■2
P1■6■12■12■6

Average WT = 2.67
Average TAT = 6.67
```

How to explain the result: Order: P2 → P3 → P1.

3. Priority Scheduling (Non-preemptive)

Concept: Select the highest-priority process.

Sample input: Input: P1 BT=5 Priority=3; P2 BT=2 Priority=1; P3 BT=4 Priority=2

C Program

```
#include <stdio.h>

int main()
{
    int n, i, j, temp;
    int bt[10], pr[10], pid[10], ct[10], tat[10], wt[10];
    float awt = 0, atat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        pid[i] = i + 1;
        printf("Enter BT for P%d: ", i + 1);
        scanf("%d", &bt[i]);
        printf("Enter Priority for P%d (1=highest): ", i + 1);
        scanf("%d", &pr[i]);
    }

    /* Smaller priority number = higher priority */
    for (i = 0; i < n - 1; i++)
        for (j = i + 1; j < n; j++)
            if (pr[i] > pr[j])
            {
                temp=pr[i]; pr[i]=pr[j]; pr[j]=temp;
                temp=bt[i]; bt[i]=bt[j]; bt[j]=temp;
                temp=pid[i]; pid[i]=pid[j]; pid[j]=temp;
            }

    ct[0] = bt[0];
    for (i = 1; i < n; i++)
        ct[i] = ct[i - 1] + bt[i];

    for (i = 0; i < n; i++)
    {
        tat[i] = ct[i];       // AT = 0
        wt[i] = tat[i] - bt[i];
        awt += wt[i];
        atat += tat[i];
    }

    printf("\nP\tBT\tPriority\tCT\tTAT\tWT\n");
    for (i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t\t%d\t%d\t%d\n",
               pid[i], bt[i], pr[i], ct[i], tat[i], wt[i]);

    printf("\nAverage WT = %.2f\n", awt / n);
    printf("Average TAT = %.2f\n", atat / n);

    return 0;
}
```

Compile and execute

```
gcc priority.c -o priority
./priority
```

Correct sample output:

```
Output:
P■BT■Priority■CT■TAT■WT
P2■2■1■■2■2■0
P3■4■2■■6■6■2
P1■5■3■■11■11■6

Average WT = 2.67
Average TAT = 6.33
```

How to explain the result: Order: P2 → P3 → P1. Priority 1 is highest.

4. SRTF — Shortest Remaining Time First (Preemptive)

Concept: Always run the available process with the shortest remaining time; a new shorter process can preempt the current one.

Sample input: Input: n=3; P1 AT=0 BT=7; P2 AT=2 BT=4; P3 AT=4 BT=1

C Program

```
#include <stdio.h>

int main()
{
    int n, i, time = 0, done = 0, shortest;
    int at[10], bt[10], rt[10], ct[10], tat[10], wt[10];
    float awt = 0, atat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("Enter AT and BT for P%d: ", i + 1);
        scanf("%d%d", &at[i], &bt[i]);
        rt[i] = bt[i];
    }

    while (done < n)
    {
        shortest = -1;

        for (i = 0; i < n; i++)
            if (at[i] <= time && rt[i] > 0)
                if (shortest == -1 || rt[i] < rt[shortest])
                    shortest = i;

        if (shortest == -1)
        {
            time++;
            continue;
        }

        rt[shortest]--;
        time++;

        if (rt[shortest] == 0)
        {
            ct[shortest] = time;
            done++;
        }
    }

    printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
    for (i = 0; i < n; i++)
    {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        awt += wt[i];
        atat += tat[i];

        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }

    printf("\nAverage WT = %.2f\n", awt / n);
    printf("Average TAT = %.2f\n", atat / n);

    return 0;
}
```

Compile and execute

```
gcc srtf.c -o srtf
./srtf
```

Correct sample output:

```
Output:
P■AT■BT■CT■TAT■WT
```

```
P1■0■7■12■12■5
P2■2■4■7■5■1
P3■4■1■5■1■0

Average WT = 2.00
Average TAT = 6.00
```

How to explain the result: Execution idea: P1 starts → P2 arrives and preempts P1 → P3 arrives and runs → P2 → P1.

5. Round Robin (Preemptive)

Concept: Give each process a fixed time quantum in cyclic order.

Sample input: Input: n=3; BT: 5 4 2; Time Quantum=2

C Program

```
#include <stdio.h>

int main()
{
    int n, i, tq, time = 0, done;
    int bt[10], rt[10], ct[10], tat[10], wt[10];
    float awt = 0, atat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("Enter Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    done = 0;

    while (done < n)
    {
        for (i = 0; i < n; i++)
        {
            if (rt[i] > 0)
            {
                if (rt[i] > tq)
                {
                    time += tq;
                    rt[i] -= tq;
                }
                else
                {
                    time += rt[i];
                    rt[i] = 0;
                    ct[i] = time;
                    done++;
                }
            }
        }
    }

    printf("\nP\tBT\tCT\tTAT\tWT\n");
    for (i = 0; i < n; i++)
    {
        tat[i] = ct[i];       // AT = 0
        wt[i] = tat[i] - bt[i];
        awt += wt[i];
        atat += tat[i];

        printf("P%d\t%d\t%d\t%d\t%d\n",
               i + 1, bt[i], ct[i], tat[i], wt[i]);
    }

    printf("\nAverage WT = %.2f\n", awt / n);
    printf("Average TAT = %.2f\n", atat / n);

    return 0;
}
```

Compile and execute

```
gcc rr.c -o rr
./rr
```

Correct sample output:

```
Output:
P■BT■CT■TAT■WT
```

```
P1■5■11■11■6
P2■4■10■10■6
P3■2■6■6■4

Average WT = 5.33
Average TAT = 9.00
```

How to explain the result: Gantt order: P1(0-2) → P2(2-4) → P3(4-6) → P1(6-8) → P2(8-10) → P1(10-11).

Student Practice — Recommended Order

- Run FCFS first and manually calculate CT, TAT and WT.
- Run SJF and explain why the shortest job is selected.
- Run Priority and explain the priority-number convention.
- Run SRTF and observe preemption when a shorter remaining job becomes available.
- Run Round Robin and change the time quantum to see how the order changes.
- Draw a Gantt chart for each sample input and verify the program output.

Important: These programs are teaching versions. They use simple assumptions to make the scheduling logic easy to understand. Tie-breaking and more complex arrival-time cases can be added after students understand the basic algorithms.